gemstone-py Examples Guide

A tour of the examples tree with diagrams and screenshots

Generated from the Markdown sources in gemstone-py/docs
Assets are local SVG illustrations stored in the repository.

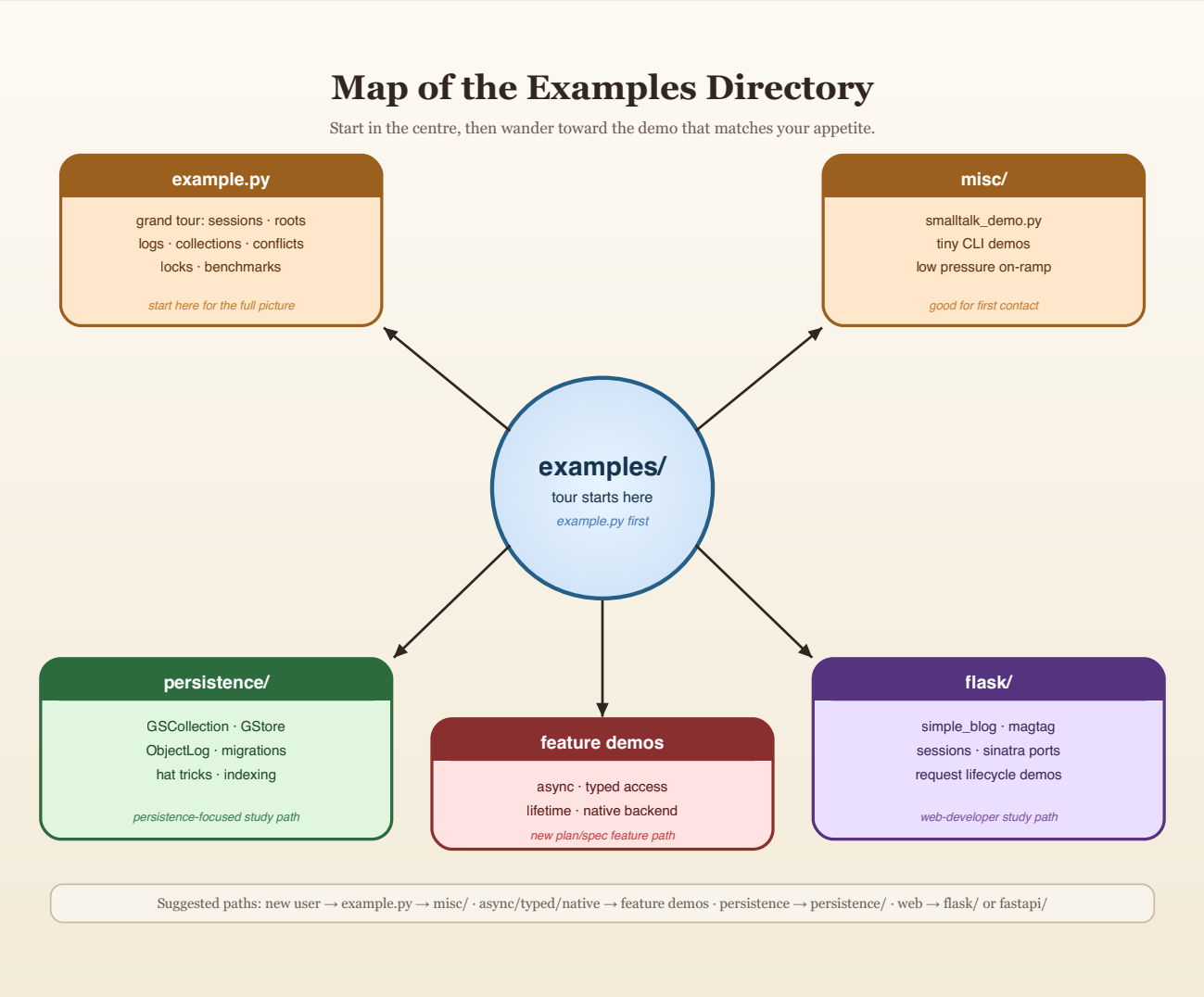
Contents

- Guide to the Examples
  - How to Use This Guide
  - Common Setup
  - Running Examples From VS Code
  - `examples/quickstart.py`: The First Live Check
  - `examples/cookbook/`
  - `examples/example.py`: The Grand Tour
  - `examples/misc/`
  - `examples/async_features/`
  - `examples/typed_access/`
  - `examples/lifetime/`
  - `examples/native_backend/`
  - `examples/persistence/`
    - Indexing
    - `GStore`
    - Hat Trick
    - Migrations
  - `examples/flask/`
    - `simple_blog`
    - `magtag`
    - `sessions`
  - `examples/fastapi/`
  - `examples/litestar/`
  - `examples/django/` and `examples/webstack/`
  - Examples vs Maintained Lanes
  - Suggested Study Paths
    - Path 1: New user
    - Path 2: Persistence-focused user
    - Path 3: Async or FastAPI user
    - Path 4: Typed API user
    - Path 5: Web developer

Path 6: Concurrency-curious person
Path 7: Packaging and performance user
Practical Advice
Closing Thought

Guide to the Examples

The `examples/` tree is one of the best parts of `gemstone-py`. It is broad enough to teach the package properly, but disciplined enough that the examples still map to the real API and current workflows.



How to Use This Guide

This guide is organized around what you are trying to learn:

- first contact with a live GemStone session
- async sessions and FastAPI request integration
- typed OOPs, typed queries, and export-set lifetime handles
- persistence helpers
- bulk root/dictionary access and explicit value conversion

- indexed collections and query-style access
- concurrency primitives
- Flask integration
- optional native backend selection
- web app translations
- maintained benchmarks versus teaching examples

If you want the shortest path:

1. run `examples/quickstart.py`
2. run `examples/example.py`
3. run `examples/misc/smalltalk_demo.py`
4. run one feature example from `async_features/`, `typed_access/`, or `lifetime/`
5. inspect one persistence example or web example

Common Setup

Most live examples expect the same GemStone environment:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

`GS_STONE_NAME` is accepted as a stone-name alias when `GS_STONE` is absent. Setting both to the same value is harmless and keeps the examples, VS Code workbench, and database explorer aligned.

Run examples as modules from the repository root so imports resolve exactly as they do in CI:

```
cd /path/to/gemstone-py
source .venv/bin/activate
python -m examples.quickstart
python -m examples.misc.smalltalk_demo
python -m examples.async_features.session_root_and_collection
```

If your prompt is in the parent directory, for example `src %`, change into the `gemstone-py` checkout first. The installed `gemstone-py` wheel exposes `gemstone-examples`, `gemstone-fastapi-example`, and `gemstone-litestar-example` from any directory; the repository-only `examples.*` modules require the checkout root on `PYTHONPATH`.

For a fresh or shared stone, you can also audit and initialize the GemStone-side roots that the persistence helpers use:

```
gemstone-bootstrap --status
gemstone-bootstrap
```

That initializes `GStoreRoot` for `GStore`, `GSQueryRoot` for `GSCollection`, and a `GemstonePyBootstrapVersion` marker. It is safe to rerun because it does not replace existing roots.

For a normal installed package:

```
python -m pip install gemstone-py
```

For a source checkout with development tools:

```
python -m pip install -e .[dev]
```

For the optional native backend example:

```
python -m pip install "gemstone-py[fast]"
```

For FastAPI examples from an installed package:

```
python -m pip install "gemstone-py[fastapi]"
gemstone-fastapi-example --reload
```

For Django examples or applications:

```
python -m pip install "gemstone-py[django]"
```

For Litestar examples or applications:

```
python -m pip install "gemstone-py[litestar]"
gemstone-litestar-example --reload
```

For FastAPI examples from a source checkout:

```
python -m pip install -e ".[examples]"
python -m examples.fastapi.run --reload
```

For Litestar examples from a source checkout:

```
python -m pip install -e ".[examples]"
python -m examples.litestar.run --reload
```

For a compact map of the broader example set:

```
gemstone-examples list
gemstone-examples plan3-map
gemstone-examples value-converters
```

Running Examples From VS Code

The repository now includes `vscode-gemstone-py-workbench/`, a companion VS Code extension scaffold for the Python-facing workflow. It adds a GemStone Py sidebar with example runners, environment/backend checks, docs/PDF launchers, maintainer scripts, Jasper links, setup verification actions, and launcher plus webview commands for `python-gemstone-database-explorer`.

For generated wrappers, run `GemStone: Open Codegen Explorer`. It connects to the configured database explorer, browses live dictionaries/classes/protocols/ methods/source, lets you select wrapper targets, previews generated files in a temporary directory, diffs them against the configured output package, saves a reusable `codegen-workbench.json` mapping, and can run the Codegen check, generation, FastAPI demo, and live target probe from the same view.

Install it from the Visual Studio Marketplace:

```
code --install-extension unicompute.gemstone-py-workbench
```

Marketplace page: https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-py-workbench

During extension development:

```
cd vscode-gemstone-py-workbench
npm install
npm run compile
```

Then open the extension folder in VS Code and press `F5`. Configure the `gemstonePy.env` settings with the same `GS_*` values shown above before running live examples. Leave `gemstonePy.repoPath` empty when the current workspace is the `gemstone-py` checkout, and set `gemstonePy.explorerPath` to a local `python-gemstone-database-explorer` checkout when you want explorer commands.

`examples/quickstart.py`: The First Live Check

Start here when you want the shortest path from environment variables to a confirmed GemStone round trip.

```
python -m examples.quickstart
gemstone-examples quickstart
```

It does three things:

- connects with `GemStoneConfig.from_env()`
- verifies the session with `3 + 4`
- stores a small dictionary under `UserGlobals` through `PersistentRoot`

This is the example to copy when you are creating a new application skeleton. The other examples are cookbook entries for specific features.

`examples/cookbook/`

This directory is the stable cookbook index. The runnable examples keep their historical paths, and `examples/cookbook/README.md` points each topic to the right script, package, or installed command.

```
gemstone-examples list
gemstone-examples plan3-map
gemstone-examples value-converters
```

Use it when you know the kind of task you want but do not yet know which example directory owns it.

The `value-converters` entry point is intentionally offline. It previews the explicit converter registry and dataclass-to-dict helper without opening a live GemStone session, which makes it a useful first check for teams deciding how much conversion they want at application boundaries.

`examples/example.py`: The Grand Tour

If you only run one broad example after the quickstart, run this one.

Why it matters:

- it exercises the major package surfaces in one file
- it demonstrates live session behaviour instead of abstract promises
- it shows transaction boundaries, cross-session reads, and conflict patterns

It covers:

- `SmalltalkBridge`
- `GemStoneSessionFacade`
- `PersistentRoot`
- bulk `PersistentRoot` and selector-send helpers
- `GStore`
- `ObjectLog`
- `RCCounter`
- `RCHash`
- `RCQueue`
- nested transactions
- `CommitConflictError`
- date/time conversions
- object locking
- instance listing

Treat it as the "here is what the package can really do" demo.

Usage:

```
python -m examples.example
```

The core shape is still ordinary session code:

```
from gemstone_py import GemStoneConfig, GemStoneSession, TransactionPolicy
from gemstone_py.session_facade import GemStoneSessionFacade

config = GemStoneConfig.from_env()

with GemStoneSession(
    config=config,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
) as session:
    facade = GemStoneSessionFacade(session)
    facade["ExampleDict"] = {"status": "ok"}
```

`examples/misc/`

This is the low-pressure on-ramp.

Start here when:

- you want a tiny first success
- you want to confirm the environment is healthy
- you want a bridge demo without reading a hundred moving parts

The star here is `smalltalk_demo.py`, which now runs through the supported CLI surface and shows the Smalltalk bridge without dragging in everything else.

Usage:

```
python -m examples.misc.smalltalk_demo
gemstone-smalltalk-demo
gemstone-examples smalltalk-demo
```

Representative code:

```
from gemstone_py.example_support import MANUAL_POLICY, example_session
from gemstone_py.smalltalk_bridge import SmalltalkBridge

with example_session(transaction_policy=MANUAL_POLICY) as session:
    smalltalk = SmalltalkBridge(session)
    settings = smalltalk.StringKeyValueDictionary.new()
    settings["status"] = "ok"
    print(settings["status"])
```

`examples/async_features/`

This directory demonstrates the async API added for modern Python application stacks.

`session_root_and_collection.py` shows:

- `AsyncSession.connect(...)`
- `async with session.transaction()`
- `AsyncPersistentRoot`
- `AsyncManagedOop`
- `AsyncGSCollection`

Run it when the normal sync login works and you want to confirm that GCI work is being routed through the async facade correctly.

Usage:

```
python -m examples.async_features.session_root_and_collection
```

Session and root access:

```
from gemstone_py.aio import AsyncPersistentRoot, AsyncSession

async with AsyncSession.connect(config=config) as session:
    async with session.transaction():
        root = AsyncPersistentRoot(session)
```

```
        await root.set("AsyncExample", {"status": "async"})

    saved = await session.root().get("AsyncExample")
```

Async managed OOPs:

```
ref = await session.execute_managed("OrderedCollection new")
try:
    first = await session.new_string("from async")
    await ref.send("add:", first)
    print(await ref.print_string())
finally:
    await ref.close()
```

Async indexed collection access:

```
from gemstone_py.aio import AsyncGSCollection

collection = AsyncGSCollection("AsyncFeaturePeople", config=config)
await collection.bulk_insert([
    {"@name": "Ada", "@status": "active"},
    {"@name": "Grace", "@status": "inactive"},
])
await collection.add_index("@status")
active = await collection.search("@status", "eql", "active")
collection.close()
```

For larger async result sets, use the async iterator:

```
async for record in collection.search_iter("@status", "eql", "active",
chunk_size=500):
    print(record["@name"])
```

Use a generated or test-specific collection name in real scripts, and call `AsyncGSCollection.drop(...)` during cleanup.

`examples/typed_access/`

This directory covers the static typing stream from the plan.

Use it when you want to understand:

- `@gemstone_class(...)`
- `TypedOop[T]`
- `typed_oop(...)`
- typed `GSCollection.query(Protocol)` predicates

- chunked `Query.iter(...)` result handling
- `gemstone-codegen` generated wrappers for method-shaped Smalltalk access
- generated wrapper `.pyi` stubs and lightweight runtime metadata

`simple_blog_queries.py` contains importable helper functions for the blog data shape. `typed_oops_and_queries.py` is the runnable live example. `codegen_demo/` shows the Protocol-to-generated-wrapper workflow.

Usage:

```
python -m examples.typed_access.typed_oops_and_queries
python -m examples.typed_access.codegen_demo.preview
gemstone-codegen \
  --module examples.typed_access.codegen_demo.models \
  --output examples/typed_access/codegen_demo/generated \
  --check
python -m examples.typed_access.codegen_demo.run --reload
python -m examples.typed_access.codegen_demo.live_probe --booking-id B-1001
```

Typed OOPs:

```
from typing import Protocol
from gemstone_py import GemStoneSession, TransactionPolicy, gemstone_class

@gemstone_class("Date")
class GemStoneDate(Protocol):
    @property
    def printString(self) -> str:
        ...

with GemStoneSession(
    config=config,
    transaction_policy=TransactionPolicy.ABORT_ON_EXIT,
) as session:
    today = session.execute_typed("Date today", GemStoneDate)  # type: ignore[type-
abstract]
    print(today.gemstone_class_name)
    print(today.proxy().printString)
```

Typed `GSCollection` queries:

```
from typing import Protocol
from gemstone_py.gsquery import GSCollection

class BlogPostRecord(Protocol):
    title: str
    status: str

posts = GSCollection("SimplePosts", config=config)
published = posts.query(BlogPostRecord).where(
```

```
    lambda post: post.status == "published"
).all()

for post in published:
    print(post.title)
```

The lambda records a GemStone ivar path. It is not a Python-side row filter.

Generated wrappers:

```
from typing import Protocol
from gemstone_py import GemStoneConfig, GemStoneSession, gemstone_class,
gemstone_selector

@gemstone_class("OkzBooking")
class OkzBookingProto(Protocol):
    status: str

    @classmethod
    @gemstone_selector("findById:")
    def find_by_id(cls, booking_id: str) -> "OkzBookingProto":
        ...

from examples.typed_access.codegen_demo.generated import OkzBooking

with GemStoneSession(config=GemStoneConfig.from_env()) as session:
    booking = OkzBooking.find_by_id(session, "B-1001")
    print(booking.status)
```

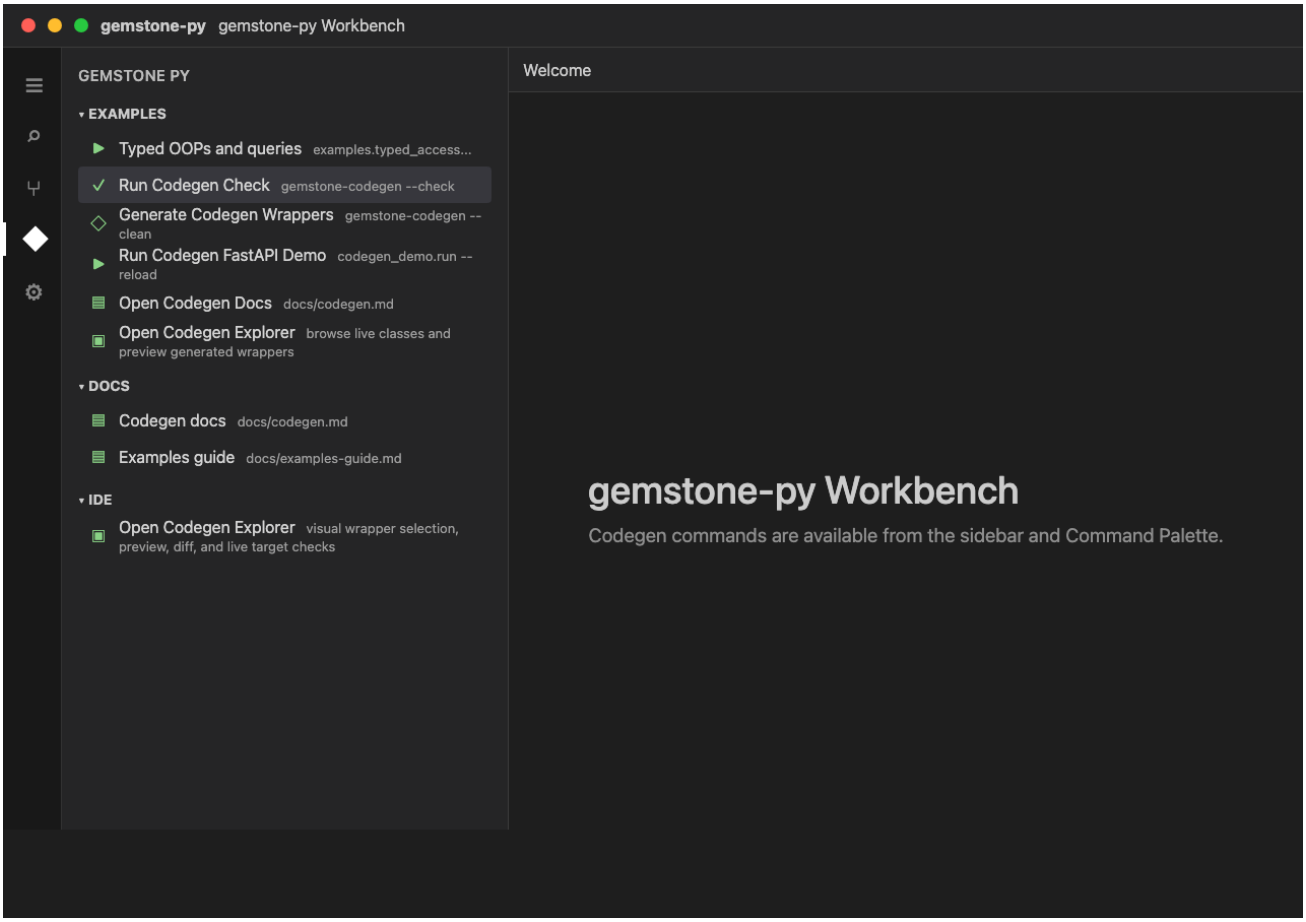
Use `@gemstone_selector(...)` for selectors that do not map cleanly from Python snake_case to Smalltalk camelCase keywords.

Generated wrappers expose `__gemstone_protocol__` and `__gemstone_selectors__`, so examples and tools can explain which Protocol and selector mapping produced a Python method without adding a runtime mapper.

The offline preview example is the safest place to start. It generates the `OkzBooking` and `OkzCustomer` wrappers in a temporary directory, shows the generated package layout, and prints the exact `gemstone-codegen --clean` and `--check` commands:

```
python -m examples.typed_access.codegen_demo.preview --show-source
```

The VS Code workbench can then make the live-class selection concrete:



Open `GemStone: Open Codegen Explorer`, connect to `python-gemstone-database-explorer`, select `OkzBooking` and `OkzCustomer`, preview the generated files, diff them against `examples/typed_access/codegen_demo/generated`, then save the selection. The example mapping file records the selectors to select from the live stone:

```
examples/typed_access/codegen_demo/codegen-workbench.example.json
```

`examples/lifetime/`

`managed_oop_handles.py` is the focused example for the OOP lifetime model.

It demonstrates:

- `execute_managed(...)` for automatic export-set retention
- `ManagedOop.close()` for explicit cleanup
- `session.handle(raw_oop)` for scoped retention when you already have an OOP

Read this before holding raw OOP integers across operations that might trigger GemStone-side garbage collection.

Usage:

```
python -m examples.lifetime.managed_oop_handles
```

Managed handle:

```
collection = session.execute_managed("OrderedCollection new")
try:
    collection.send("add:", session.new_string("managed"))
    session.eval("System startGcAndCommit")
    print(collection.print_string())
finally:
    collection.close()
```

Scoped raw-OOP handle:

```
raw_oop = session.execute_oop("OrderedCollection new")
with session.handle(raw_oop) as handle:
    handle.send("add:", session.new_string("scoped"))
    print(handle.send("printString"))
```

Use `execute_managed(...)` for object references you want Python to retain automatically. Use `session.handle(...)` when an existing raw OOP only needs to be protected inside a known block.

`examples/native_backend/`

`check_backend.py` prints the selected low-level GCI backend and whether the optional native extension is importable.

Use it after:

```
python -m pip install "gemstone-py[fast]"
```

It is also useful when comparing benchmark reports with `GEMSTONE_PY_GCI_BACKEND=ctypes` and `GEMSTONE_PY_GCI_BACKEND=native`.

Usage:

```
python -m examples.native_backend.check_backend
GEMSTONE_PY_GCI_BACKEND=ctypes python -m examples.native_backend.check_backend
GEMSTONE_PY_GCI_BACKEND=native python -m examples.native_backend.check_backend
```

The code is deliberately tiny:

```
from gemstone_py import _gci
from gemstone_py.native import native_fast_path_available

print(_gci.IMPLEMENTATION)
print(native_fast_path_available())
```

Backend selection happens when Python imports `gemstone_py._gci`, so set `GEMSTONE_PY_GCI_BACKEND` before starting Python.

`examples/persistence/`

This is where the package stops being a client library and starts feeling like a practical application toolkit.

Major themes in this tree:

- indexed collections
- stores
- ordered collections and batch updates
- data migration
- persistent data structures
- translation of old MagLev-era ideas into plain GemStone use

Indexing

Files around `persistence/indexing/` show how to:

- create a dataset
- store rows in GemStone
- build indexes
- search efficiently

This is the right cluster to study when `PersistentRoot` starts to feel too coarse-grained for the shape of your data.

Usage:

```
python -m examples.persistence.indexing.create_random_people
python -m examples.persistence.indexing.search_random_people
python -m examples.persistence.indexing.index_example
```

The useful API shape:

```
from gemstone_py.gsquery import GSCollection
```

```
people = GSCollection("People", config=config)
people.bulk_insert([
    {"@name": "Ada", "@age": 36},
    {"@name": "Grace", "@age": 42},
])
people.add_index_for_class("@age", "SmallInt")
matches = people.search("@age", "gte", 40)
```

GStore

The `persistence/gstore/` example contrasts GemStone-backed storage with an in-memory dict baseline. That makes it useful for both teaching and performance intuition:

- you see the API shape
- you see the cost of persistence
- you do not need mythology to explain the result

Usage:

```
python -m examples.persistence.gstore.main
```

Typical store-shaped code:

```
from gemstone_py.gstore import GStore

store = GStore("inventory.db", config=config)
store["sku:hat"] = {"name": "Hat", "stock": 8}
print(store["sku:hat"])
```

Hat Trick

The `hat_trick/` example is the kind of thing every useful repository-backed toolkit should have: a weird little demo that is memorable enough to teach the real abstraction.

Here the abstraction is queue-backed shared state:

- create a hat backed by `RCQueue`
- inspect the contents
- understand the relation between a playful example and a real concurrency primitive

The queue is not a toy. The hat is merely wearing formal attire.

Usage:

```
python -m examples.persistence.hat_trick.create_hat
python -m examples.persistence.hat_trick.add_rabbit_to_hat
python -m examples.persistence.hat_trick.show_hat_contents
```

The pattern behind the example is reduced-conflict queue-backed state:

```
from gemstone_py.concurrency import RCQueue
from gemstone_py.persistent_root import PersistentRoot

root = PersistentRoot(session)
root["WorkQueue"] = RCQueue(session)
root["WorkQueue"].push("first job")
```

Migrations

The migrations examples are valuable because they show the package in an honest "old data must become new data" mode.

Use these when you want patterns for:

- versioned domain objects
- chunked migration
- retry loops
- explicit transactional migration steps
- module-style forward and rollback migrations

Usage:

```
python -m examples.persistence.migrations.write_posts
python -m examples.persistence.migrations.migrate
python -m examples.persistence.migrations.migrate_by_chunks
```

For application migrations, scaffold a numbered file:

```
gemstone-migrations scaffold add_amount_to_booking --directory migrations
```

That creates a module with `upgrade(session)` and `downgrade(session)`. Register files in a manifest:

```
migrations = [
    "my_app.migrations.001_initial",
    "my_app.migrations.002_add_amount_to_booking",
]
```

Then apply or dry-run the manifest:

```
from gemstone_py import GemStoneConfig, GemStoneSession
from gemstone_py.migrations import current_version, load_manifest, upgrade

steps = load_manifest("my_app.migrations.manifest")

with GemStoneSession(config=GemStoneConfig.from_env()) as session:
    print(current_version(session))
    print(upgrade(session, steps, dry_run=True))
    upgrade(session, steps)
```

Example migration body:

```
id = "002_add_amount_to_booking"
dependencies = ("001_initial",)

def upgrade(session):
    session.eval(
        "OkzBooking addInstVarName: 'amount' ifAbsent: [ nil ]"
    )
    session.commit()

def downgrade(session):
    session.eval(
        "OkzBooking removeInstVarName: 'amount' ifAbsent: [ nil ]"
    )
    session.commit()
```

The same workflow is available from the command line:

```
gemstone-migrations current
gemstone-migrations status --manifest my_app.migrations.manifest
gemstone-migrations plan --manifest my_app.migrations.manifest
gemstone-migrations upgrade --manifest my_app.migrations.manifest --dry-run
gemstone-migrations upgrade --manifest my_app.migrations.manifest --dry-run --record
gemstone-migrations upgrade --manifest my_app.migrations.manifest
gemstone-migrations downgrade --manifest my_app.migrations.manifest --target
001_initial
```

Before applying a new step, the runner checks that every GemStone-applied migration id still exists in the local manifest and that stored checksums still match local migration files when checksums are available.

Real `upgrade` and `downgrade` runs also acquire an advisory lock in `UserGlobals` before applying steps. Dry-runs do not acquire the lock. If a previous process crashed and left a stale lock, use a reviewed override:

```
gemstone-migrations upgrade --manifest my_app.migrations.manifest \
  --force-lock --lock-owner release-2026-05-11
```

Use `--no-lock` only for controlled maintenance windows where another guard is already preventing concurrent migration runs.

Plain `--dry-run` prints the planned migration ids. Add `--record` when the migration body is mostly direct session calls such as `session.eval(...)`; the runner executes callbacks against a recording session and prints the calls it would have made without sending them to GemStone.

Recorded output is intentionally plain so it can be reviewed in a release log:

```
dry-run upgrade: 1 step(s)
  002_add_amount_to_booking
recorded operations:
  # upgrade 002_add_amount_to_booking
  session.eval("OkzBooking addInstVarName: 'amount' ifAbsent: [ nil ]")
  session.commit()
```

If a migration branches on live query results, use the recorded dry-run as a partial review aid only. The recorder returns harmless placeholders for OOPs and does not ask GemStone for real state.

When a local Protocol or type witness has drifted from a GemStone class, compare the two before writing a migration:

```
gemstone-migrations diff-class OkzBooking \
  --local-class my_app.models:OkzBookingProto
```

The command prints missing and extra instance variables plus advisory `session.eval(...)` lines that can be copied into a reviewed migration.

For startup or release checks, fingerprint the expected roots, classes, and migration state:

```
gemstone-migrations fingerprint \
  --root Bookings \
  --class OkzBooking \
  --manifest my_app.migrations.manifest \
  --json
```

The key habit is to keep migration units explicit:

```
from gemstone_py import session_scope

with session_scope(config=config) as session:
    # read old shape
    # write new shape
```

```
    # commit happens only if the block succeeds
    ...
```

examples/flask/

This directory is where the request/session layer stops being theory.

Important sub-examples:

- `simple_blog/`
- `magtag/`
- `sessions/`
- `sinatra_port/`

simple_blog

Good first Flask example because:

- the domain is small
- the persistence is recognizable
- the routes are ordinary enough to reason about quickly

Study this when you want:

- request session handling
- a simple persistent model
- proof that the package fits normal Flask structure

Usage:

```
python -m examples.flask.simple_blog.blog_app
```

Request-scoped work should use the installed session integration:

```
from flask import Flask
from gemstone_py import GemStoneConfig, install_flask_request_session

app = Flask(__name__)
install_flask_request_session(
    app,
    config=GemStoneConfig.from_env(),
    pool_size=4,
)
```

magtag

This is a larger demonstration of the same core idea:

- Flask app
- GemStone-backed data
- `GSCollection` in a realistic setting

If `simple_blog` is the friendly coffee, `magtag` is the "all right, show me a real screen and some real workflows" example.

Usage:

```
python -m examples.flask.magtag.magtag_app
```

The model layer uses `GSCollection` for lookup-heavy records:

```
from gemstone_py.gsquery import GSCollection

users = GSCollection("MagTagUsers")
users.add_index_for_class("@name", "String")
matches = users.search("@name", "eql", "pbm0")
```

sessions

These examples focus on request/session lifecycle concerns. They are useful when you care more about integration behaviour than about application domain logic.

Usage:

```
python -m examples.flask.sessions.msessions
```

Provider health can be inspected without reaching into private Flask state:

```
from gemstone_py import flask_request_session_provider_snapshot

snapshot = flask_request_session_provider_snapshot(app)
if snapshot is not None:
    print(snapshot.available, snapshot.in_use, snapshot.created)
```

examples/fastapi/

This is the smallest async web example. It uses `session_dependency(...)` to open one `AsyncSession` per request and commit or abort with the request outcome.

Usage:

```
python -m pip install -e ".[examples]"
python -m examples.fastapi.run --reload
```

When the server starts, you should see output like:

```
INFO:     Will watch for changes in these directories: ['/path/to/gemstone-py']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [49045] using WatchFiles
INFO:     Started server process [49048]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

With that server running, test it from a second terminal.

Basic checks:

```
curl -i http://127.0.0.1:8000/
```

Expected:

```
HTTP/1.1 200 OK
```

Body should include:

```
{"name":"gemstone-py FastAPI example","endpoints":{"health":"/health/
gemstone","docs":"/docs","openapi":"/openapi.json"}}
```

Then test the GemStone endpoint:

```
curl -i http://127.0.0.1:8000/health/gemstone
```

Expected if GemStone credentials/environment are set and the stone is reachable:

```
{"result":7}
```

Also open these in a browser:

```
http://127.0.0.1:8000/
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/health/gemstone
```

Endpoint shape:

```
from fastapi import Depends, FastAPI
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession, AsyncSessionPool
from gemstone_py.aio.fastapi import pool_session_dependency, session_dependency

app = FastAPI()
get_gemstone_session = session_dependency(config=GemStoneConfig.from_env())

@app.get("/health/gemstone")
async def gemstone_health(session: AsyncSession = Depends(get_gemstone_session)):
    return {"result": await session.eval("3 + 4")}
```

For production-style request reuse, use a pool-backed dependency:

```
pool = AsyncSessionPool(maxsize=8, minsize=2, config=GemStoneConfig.from_env())
get_gemstone_session = pool_session_dependency(pool)
```

`examples/litestar/`

This is the smallest Litestar web example. It uses `gemstone_py.aio.litestar.session_dependency(...)` to open one `AsyncSession` per request and finalize it with the request outcome.

Usage:

```
python -m pip install -e ".[examples]"
python -m examples.litestar.run --reload
```

The root route returns the available endpoints:

```
{"name":"gemstone-py Litestar
example","framework":"Litestar","adapter":"gemstone_py.aio.litestar.session_dependenc
y","dependencyInjection":"litestar.di.Provide","endpoints":{"health":"/health/
gemstone","docs":"/schema/swagger","openapi":"/schema/openapi.json"}}
```

This deliberately mirrors the FastAPI example's `/` and `/health/gemstone` contract while showing the Litestar-specific adapter, `Provide(...)` dependency injection, and schema routes.

The installed package runner is:

```
gemstone-litestar-example --reload
```

`examples/django/` and `examples/webstack/`

These matter for two reasons:

1. they prove the package is not locked into one tiny application style
2. they surface compatibility issues that smaller demos never trigger

The `webstack` examples are also useful for release verification because they have enough app surface to catch missing imports, route regressions, and other "this only broke in CI" problems.

Usage:

```
cd examples/django/myapp
python manage.py migrate
python manage.py runserver
```

```
python -m examples.webstack.magtag_app
```

Examples vs Maintained Lanes

This distinction matters.

The examples are for learning and exploration.

The maintained lanes are:

- `./scripts/run_ci_checks.sh`
- `./scripts/run_live_checks.sh`
- `gemstone-benchmarks`
- the GitHub workflows under `.github/workflows/`

Do not confuse:

- "I ran a charming example"
- with
- "the package is verified against its supported workflows"

You need both. They are not interchangeable.

Suggested Study Paths

Path 1: New user

1. `examples/quickstart.py`
2. `examples/example.py`
3. `examples/misc/smalltalk_demo.py`
4. this guide again, now with less fear

Path 2: Persistence-focused user

1. `examples/quickstart.py`
2. `examples/example.py`
3. `examples/persistence/indexing/*`
4. `examples/persistence/gstore/main.py`
5. `examples/persistence/migrations/*`

Path 3: Async or FastAPI user

1. `examples/async_features/session_root_and_collection.py`
2. `examples/fastapi/app.py`
3. `tests/test_live_async_integration.py`

Path 4: Typed API user

1. `examples/typed_access/typed_oops_and_queries.py`
2. `examples/typed_access/simple_blog_queries.py`
3. `tests/test_typed_oop_lifetime.py`
4. `tests/test_gsquery.py`

Path 5: Web developer

1. `examples/flask/simple_blog/*`
2. `examples/flask/sessions/*`
3. `examples/flask/magtag/*`
4. `examples/fastapi/app.py`
5. `examples/webstack/*`

Path 6: Concurrency-curious person

1. `examples/example.py`
2. hat trick queue demo
3. live tests around contention and retries

Path 7: Packaging and performance user

1. `examples/native_backend/check_backend.py`
2. `gemstone-benchmarks --suite gci --entries 1000000`
3. `docs/performance.md`
4. `./scripts/run_native_checks.sh`

Practical Advice

- run the examples from a configured environment, not from a shell that "mostly remembers" the right variables
- do not start with the biggest web example if you still do not know how `TransactionPolicy` works
- read the maintained benchmark docs before treating example performance output as policy
- keep the examples open next to the user manual; the two reinforce each other

Closing Thought

The examples directory is not filler. It is one of the package's strongest teaching tools, and it is worth reading like source documentation instead of like a pile of promotional demos.

> This PDF was generated locally from the Markdown sources in `docs/`. If you edit the source files, rerun `python docs/build_pdf_docs.py`.